

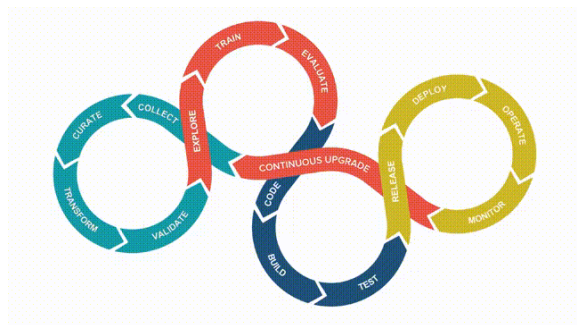


**RV College of
Engineering®**

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

Machine Learning (MLOps)



Prof. Manasa M
Assistant Professor
Dept of AIML
RVCE



Unit -4

Deployment to Production

- CI/CD Pipelines
- Building ML Artifacts
- Deployment Strategies
- Containerization
- Scaling Deployments
- Requirements and Challenges.

Feedback Loop

- How Often Should Models Be Retrained?
- Understanding Model Degradation
- Drift Detection in Practice
- The Feedback Loop

Unit -4

Deployment to Production

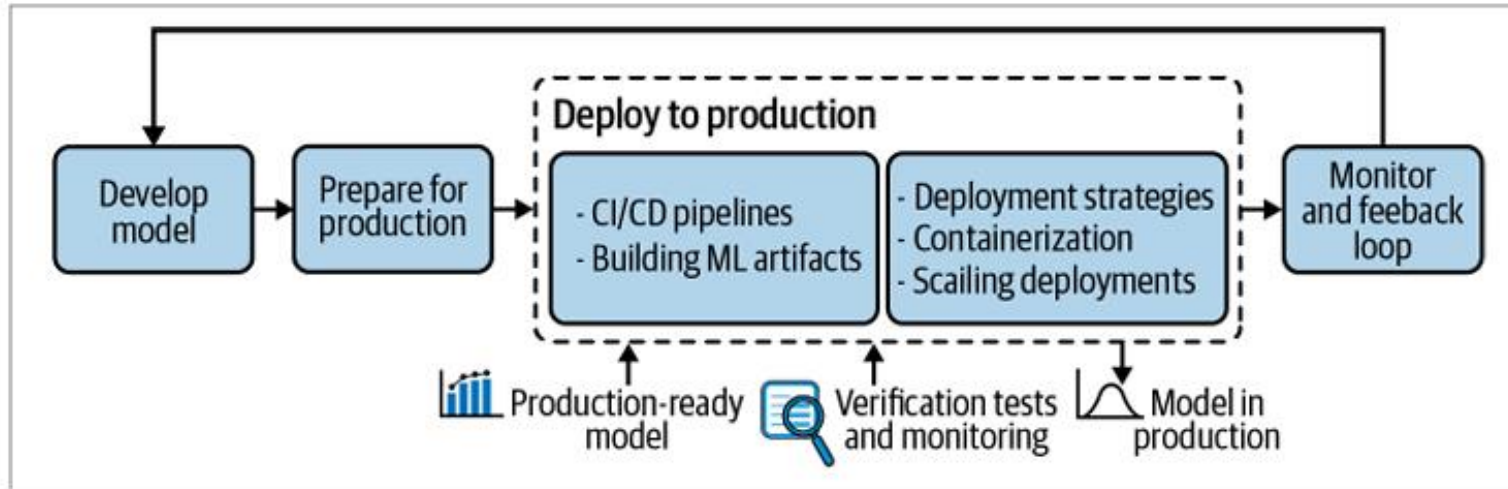


Figure 6-1. Deployment to production highlighted in the larger context of the ML project life cycle



Unit -4

- The two form a modern philosophy of agile software development and a set of practices and tools to release applications
- these concepts should be tools to serve the purpose of delivering quality fast, and the first step is always to identify the specific risks present at the organization
- CI/CD concepts apply to traditional software engineering, but they apply just as well to machine learning systems and are a critical part of MLOps strategy.
- After success fully developing a model, a data scientist should push the code, metadata, and docu mentation to a central repository and trigger a CI/CD pipeline.
- an incremental approach to building a CI/CD pipeline is preferred: a simple or even naïve workflow on which a team can iterate is often much better than starting with complex infrastructure from scratch.
- A starting project does not have the infrastructure requirements of a tech giant
- There are common tools and best practices, but there is no one-size-fits-all CI/CD methodology.
- The best path forward is starting from a simple (but fully functional) CI/CD workflow and introducing additional or more sophisticated steps along the way as quality or scaling challenges appear



Unit -4

1. Build the model
 - a. Build the model artifacts
 - b. Send the artifacts to long-term storage
 - c. Run basic checks (smoke tests/sanity checks)
 - d. Generate fairness and explainability reports
2. Deploy to a test environment
 - a. Run tests to validate ML performance, computational performance
 - b. Validate manually
3. Deploy to production environment
 - a. Deploy the model as canary
 - b. Fully deploy the model



Unit -4

Building ML Artifacts

- The goal of a continuous integration pipeline is to avoid unnecessary effort in merging the work from several contributors and detect bugs or development conflicts as soon as possible.
- The very first step is using centralized version control systems
- The most common version control system is Git, an open source software initially developed to manage the source code for the Linux kernel.
- It allows for maintaining a clear history of changes, safe rollback to a previous version of the code, multiple contributors to work on their own branches of the project before merging to the main branch, etc.
- **Git is not designed** to store other types of assets common in data science workflows, such as large binary files (for example, trained model weights), or to version the data itself.
- Data versioning is a more complex topic with numerous solutions, including Git extensions, file formats, databases, etc



Unit -4

What is ML Artifacts

- Once the code and data is in a centralized repository, a testable and deployable bundle of the project must be built.
- These bundles are usually called artifacts in the context of CI/CD.
 - Code for the model and its preprocessing
 - Hyperparameters and configuration
 - Training and validation data
 - Trained model in its runnable form
 - An environment including libraries with specific versions, environment variables, etc.
 - Documentation
 - Code and data for testing scenarios



Unit -4

Testing pipeline

- the testing pipeline can validate a wide variety of properties of the model contained in the artifact..
- In addition to verifying compliance with requirements, good tests should make it as easy as possible to diagnose the source issue when they fail.
- For that purpose, naming the tests is extremely important, and carefully choosing a number of datasets to validate the model against can be valuable.
- A test on a fixed (not automatically updated) dataset with simple data and not too-restrictive performance thresholds can be executed first and called “base case.”



Unit -4

Testing pipeline

- A number of datasets that each have one specific oddity (missing values, extreme values, etc.) could be used with tests appropriately named so that the test report immediately shows the kind of data that is likely to make the model fail.
- These datasets can represent realistic yet remarkable cases, but it may also be use ful to generate synthetic data that is not expected in production.
- an essential part of model validation is testing on recent production data. One or several datasets should be used, extracted from several time windows and named appropriately.
- This category of tests should be performed and automatically analyzed when the model is already deployed to production.



Unit -4

Testing pipeline

- Automating these tests as much as possible is essential and, indeed, is a key component of efficient MLOps.
- A lack of automation or speed wastes time, discourages the development team from testing and deploying that can delay the discovery of bugs or design choices that make it impossible to deploy to production.
- a development team can hand over a monthslong project to a deployment team that will simply reject it because it does not satisfy requirements for the production infrastructure.



Unit -4

Testing pipeline

- The most widespread tool for software engineering continuous integration is Jenkins, a very flexible build system that allows for the building of CI/CD pipelines regardless of the programming language, testing framework, etc.
- Jenkins can be used in data science to orchestrate CI/CD pipelines, although there are many other options.



Unit -4

Deployment Strategies

- **Integration** The process of merging a contribution to a central repository and performing more or less complex tests.
- **Delivery** As used in the continuous delivery (CD) part of CI/CD, the process of building a fully packaged and validated version of the model ready to be deployed to production.



Unit -4

Development Strategies

Deployment

- **The process of running a new model version on a target infrastructure.**
- Fully automated deployment is not always practical or desirable and is a business decision as much as a technical decision, whereas continuous delivery is a tool for the development team to improve productivity and quality as well as measure progress more reliably.
- Continuous delivery is required for continuous deployment, but it also provides enormous value without



Unit -4

Development Strategies

Release

In principle, release is yet another step, as deploying a model version (even to the production infrastructure) does not necessarily mean that the production work load is directed to the new version.

Multiple versions of a model can run at the same time on the production infrastructure.



Unit -4

Categories of Model Deployment

- Batch scoring, where whole datasets are processed using a model, such as in daily scheduled jobs.
- Real-time scoring, where one or a small number of records are scored, such as when an ad is displayed on a website and a user session is scored by models to decide what to display.



Unit -4

Feature	Batch Scoring	Real-Time Scoring
Processing mode	Many records at once	Single/small number of records
Speed	Minutes to hours	Milliseconds
Use cases	Reports, segmentation, credit scoring	Ads, fraud detection, recommendations
Staleness	Predictions may be old	Always fresh
Cost	Lower, bulk compute	Higher, always-on API



Unit -4

Maintenance in Production

Resource monitoring - collecting IT metrics such as CPU, memory, disk, or network usage can be useful to detect and troubleshoot issues.

Health check - To check if the model is online and to analyze its latency, it is common to implement a health check mechanism that simply queries the model at a fixed interval (on the order of one minute) and logs the results.

ML metrics monitoring - Analyzing the accuracy of the model and comparing it to another version or detecting when it is going stale.

Since it may require heavy computation, this is typically lower frequency, but as always, will depend on the application; it is typically done once a week.



Unit -4

Containerization

- Deploying to production should automatically and reliably rebuild this environment on the target machine.
- In addition, the target machine will typically run multiple models simultaneously, and two models may have incompatible dependency versions.
- Finally, several models running on the same machine could compete for resources, and one misbehaving model could hurt the performance of multiple cohosted models.

Containerization tools bundle an application together with all of its related configuration files, libraries, and dependencies that are required for it to run across different operating environments.



Unit -4

Containerization

Docker : Released in 2014, it has become the de facto standard. It allows an application to be packaged, sent to a server (the Docker host), and run with all its dependencies in isolation from other applications.

When deploying a model, the framework should solve a number of issues:

- Which Docker host(s) should receive the container?
- When a model is deployed in several copies, how can the workload be balanced?
- What happens if the model becomes unresponsive, for example, if the machine hosting it fails? How can that be detected and a container reprovisioned?
- How can a model running on multiple machines be upgraded, with assurances that old and new versions are switched on and off, and that the load balancer is updated with a correct sequence?



Unit -4

Feature/Aspect	Docker	Kubernetes
Basic Definition	Platform for creating, deploying, and running applications in containers.	Open-source container orchestration platform.
Primary Purpose	Provides a consistent environment for applications using containers.	Manages, scales, and operates containers across multiple hosts.
Scale	Deals with individual containers or services.	Operates at the cluster level, across multiple machines.
Components	- Docker Engine - Docker Compose	- Nodes - Pods - Services
Networking	Uses network bridges for container communication.	Each Pod gets its own IP. Services facilitate communication.
Storage	Provides volume constructs for persistent storage.	Uses Persistent Volumes (PVs) and Persistent Volume Claims (PVCs) for more flexible storage options.
GUI	Docker Desktop and Docker Dashboard	Kubernetes Dashboard
Usage Together	Can be used as the container runtime for Kubernetes.	Can orchestrate and manage containers created by Docker.



Unit -4

Scaling Deployment

Two types of growth challenges:

- The ability to use a model in production with high-scale data
- The ability to train larger and larger numbers of models
- Scalability is ideal for systems with steady, predictable growth.
- Elasticity is better suited for systems that experience sudden spikes or dips in usage.

A computational system is said to be horizontally scalable if it is possible to incrementally add more computers to expand its processing power.

Example, a Kubernetes cluster can be expanded to hundreds of machines.



Unit -4

Scaling Deployment

Kubernetes and Scaling

- Handling more data for real-time scoring is made much easier by frameworks such as Kubernetes.
- Since most of the time trained models are essentially formulas, they can be replicated in the cluster in as many copies as necessary.
- With the auto-scaling features in Kubernetes, both provisioning new machines and load balancing are fully handled by the framework
- and setting up a system with huge scaling capabilities is now relatively simple



Unit -4

Scaling Deployment

Elastic System :

- An elastic system allows, in addition to being scalable, easy addition and removal of resources to match the compute requirements.
- Example, a Kubernetes cluster in the cloud can have an auto-scaling capability that automatically adds machines when the cluster usage metrics are high and removes them when they are low.
- In principle, elastic systems can optimize the usage of resources; they automatically adapt to an increase in usage without the need to permanently provision resources that are rarely required.



Unit -4

Strategies to distribute the computation for huge volume data:

- Using a framework that handles **distributed computation** natively, Spark.
- Distribute batch processing is to **partition the data**
- In terms of computation, scaling the number of models is somewhat simpler.
- The key is to add more computing power and to make sure the monitoring infrastructure can handle the workload.
- **But in terms of governance and processes, this is the most challenging situation.**
- scaling the number of models means that the CI/CD pipeline must be able to handle large numbers of deployments.
- As the number of models grows, the need for automation and governance grows, as human verification cannot necessarily be systematic or consistent.



Unit -4

Requirements

When deploying a model, there are several possible scenarios:

- One model deployed on one server
- One model deployed on multiple servers
- Multiple versions of a model deployed on one server
- Multiple versions of a model deployed on multiple servers
- Multiple versions of multiple models deployed on multiple servers

An effective logging system should be able to generate centralized datasets that can be exploited by the model designer or the ML engineer, usually outside of the production environment.

The system can access and retrieve scoring logs from multiple servers, either in a real-time scoring use case or in a batch scoring use case.

- When a model is deployed on multiple servers, the system can handle the mapping and aggregation of all information per model across servers.
- When different versions of a model are deployed, the system can handle the mapping and aggregation of all information per version of the model across servers



Unit -4

Requirements

Logging

An effective logging system should be able to generate centralized datasets that can be exploited by the model designer or the ML engineer, usually outside of the production environment.

- The system can access and retrieve scoring logs from multiple servers, either in a real-time scoring use case or in a batch scoring use case.
- When a model is deployed on multiple servers, the system can handle the mapping and aggregation of all information per model across servers.
- When different versions of a model are deployed, the system can handle the mapping and aggregation of all information per version of the model across servers



Unit -4

Challenges

Logging

- For **large-scale machine learning applications**, the number of raw event logs generated can be an issue if there are no preprocessing steps in place to filter and aggregate data.
- For **real-time scoring use cases**, logging streaming data requires setting up a whole new set of tooling that entails a significant engineering effort to maintain.
- However, in both cases, because the goal of monitoring is usually to estimate aggregate metrics, saving only a subset of the predictions may be acceptable.



Unit -4

Challenges

Logging

- For **large-scale machine learning applications**, the number of raw event logs generated can be an issue if there are no preprocessing steps in place to filter and aggregate data.
- For **real-time scoring use cases**, logging streaming data requires setting up a whole new set of tooling that entails a significant engineering effort to maintain.
- However, in both cases, because the goal of monitoring is usually to estimate aggregate metrics, saving only a subset of the predictions may be acceptable.